

Tema 5 Lenguaje de programación PYTHON

Práctica 3 - Tipos de Datos Compuestos

Listas, tuplas, conjuntos y diccionarios

EJERCICIO 1. Pruebas Básicas con Listas

Crea un programa en el que pruebes las siguientes operaciones con listas.

Comienza creando dos listas: *lista1* con los datos 5, 8 y 10; y *lista2* con los datos 3, 2, 9, 12, y 4.

1. Usa `len()` para averiguar la longitud de cada lista.
2. Usa el operador `+` para concatenarlas, y averigua la longitud del resultado.
3. Encuentra el número mayor de ambas listas. Puedes hacerlo, bien encontrando primero el máximo de la lista **lista1**, después el máximo de la lista **lista2** y finalmente el máximo de ambos máximos, o bien encontrando el máximo de la concatenación. Prueba ambos enfoques.
4. Escribe una expresión que te de la suma del primer elemento de cada lista (debe sumar el 5 con el 3 y producir 8).
5. Escribe una expresión que sume el último elemento de cada lista (10+4). Si no conocieses las longitudes de las listas, ¿qué expresión usarías para acceder al último elemento? Pruébalo.
6. Escribe una asignación que sustituya el 8 de la lista **lista1** por un cero, comprueba que el elemento ha cambiado.
7. Ejecuta la asignación "**lista3=lista2**", e imprime **lista3** para comprobar que contiene los valores 3,2,9,12 y 4
8. Modifica **lista3**, de modo que su primer elemento valga 7. Imprímela para comprobar que así es.
9. Imprime ahora **lista2**. Verás que su primer elemento también ha cambiado. Esto se debe a que, a partir de la asignación "**lista3=lista2**" ambas variables apuntan a la misma lista.
10. Añade el número 5 al final de **lista1**. Imprime la lista para comprobar el resultado.
11. Inserta el número 99 en la posición 1 (entre el primer y segundo elemento) de **lista2**. Imprime la lista para comprobarlo.
12. Elimina el número 9 de **lista2** y verifica que ya no está.
13. Elimina el último elemento de **lista1**. Imprime el valor eliminado y la lista resultante.
14. Utiliza el método `index()` para encontrar la posición del número 10 en *lista1* y el método `count()` para saber cuántas veces aparece el número 5 en esa lista.
15. Crea una sublista que contenga sólo los elementos en posición par (0, 2, 4, ...) de **lista1**.
16. Crea una copia por valor de **lista1** llamada **copia**. Cambia el primer valor de **copia** y demuestra que **lista1** no se ha visto afectada. (Usa "**copia = list(lista1)**")

EJERCICIO 2. Listas y Conjuntos

Si a partir de una lista de elementos que contiene valores repetidos, queremos obtener otra lista en la que sólo aparezcan valores únicos (es decir, eliminar elementos repetidos), un truco

que puede usarse es aprovechar una característica fundamental de los conjuntos de Python, pues estos no contienen valores repetidos.

Crea un programa que comience con una lista:

```
valores = [4, 6, 2, 4, 9, 6, 2, 1, 10, 2, 6, 8]
```

Convierte la lista a un conjunto, el cual volverás a transformar en una lista llamada *sin_repetir* y ordénala de menor a mayor con la función `sorted()`. Obtendrás algo como:

```
sin_repetir: [1, 2, 4, 6, 8, 9, 10]
```

Crea los conjuntos de números:

- conjunto1 = {1,2,3,4,5,6}
- conjunto2 = {4,5,6,7,8,9}

Utilizando las operaciones de conjuntos, calcula e imprime en pantalla:

- Números que están en los dos conjuntos
- Números que solo están en un conjunto
- Números que solo están en el conjunto1

Pide por teclado 3 números, añádelos a un nuevo conjunto y muestra en pantalla si:

- Los 3 números están todos en alguno de los dos conjuntos.
- Los 3 números están en alguno de los dos conjuntos
- Alguno de los números está en el conjunto1
- Ninguno de los números está en conjunto1 ni en conjunto2

EJERCICIO 3. Diccionarios

Crea un programa que:

1. Solicite por teclado los nombres de tres países y sus capitales.
2. Guarde esa información en un diccionario utilizando el país como clave.
3. Muestre el diccionario por pantalla.
4. Extraiga del diccionario e imprima en pantalla lo siguiente:
 - Países almacenados
 - Capitales almacenadas
 - Lista de tuplas (país, capital) con toda la información almacenada en el diccionario
5. Pida por teclado el nombre de un país e imprima su capital utilizando **`.get()`**. Si el usuario teclea un país que no existe, se le mostrará el mensaje "País no encontrado".

EJERCICIO 4. Fichero a Listas y Tratamiento de Datos

Crea un fichero denominado "datos.txt" con información sobre distintas personas.

Cada línea contiene cinco campos separados por comas:

```
Nombre,Edad,Ciudad,Estudios,Hobby
```

Por ejemplo, el contenido podría ser:

```
Ana,21,Madrid,Ingeniería,Leer
```

```
Luis,19,Valencia,Medicina,Fútbol
```

```
Carmen,22,Sevilla,Historia,Pintar
```

```
Pedro,20,Bilbao,Economía,Ajedrez
```

```
Lucía,23,Barcelona,Biología,Viajar
```

A partir de este fichero, implementa las siguientes tareas de manera progresiva:

1. Leer todo el fichero usando el método `read()` almacenando la información en una cadena llamada *contenido* y mostrar por pantalla dicha cadena.
Observa la variable *contenido* con las líneas del fichero separadas por `'\n'` (salto de línea)
2. Crea una nueva lista *personas* troceando la cadena con el método `split()`. Imprímela.
3. Aprovechando que una lista es un iterable, utiliza la sintaxis de comprensión de listas:
`lista_personas = [persona.split(',') for persona in personas]`
Observa el resultado...
4. Accede a elementos concretos de la lista *personas* para mostrar:
 - El nombre de la primera persona.
 - La ciudad de la última persona.
 - El hobby de la tercera persona.
5. Genera dos listas, una con los nombres de las personas y otra con sus estudios
6. Generar una lista que contenga únicamente los nombres de las personas cuya edad sea mayor que 20 años. Para ello, practica el uso de:
`[ <expresión> for <elemento> in <iterable> if <condición> ]`